

PURPOSE

Origin

Together started as a personal frustration.

To make music with each other, one of us had to go to the other's house and share a computer. When that wasn't possible, we'd connect over Discord — one person sharing their screen, the other describing what to do instead of being able to do it themselves. It was slow, it was limiting, and it made the creative process feel like a support call rather than a collaboration.

We decided to build what we wished existed.

What we found

After investigating existing tools, we discovered two separate problems.

The first is collaboration. Native DAWs like Ableton and FL Studio have no real-time collaboration. You work alone or you work in the same room. Cloud-based tools like Soundtrap or BandLab offer collaboration but sacrifice depth — they are browser applications built for accessibility, not for engineering.

The second is depth of control. Every major DAW — native or cloud — treats audio as an opaque object. You can adjust its volume, apply effects to it, cut it into pieces. But you cannot reach inside it. The signal is given to you as a single primitive, and that is the only level at which you can work.

We believe both of these problems have the same root cause: performance. Current DAWs use only the CPU for their entire workload. That budget runs out fast, and the software compensates by abstracting away complexity. The deeper operations become impossible, so they are never exposed.

What we are building

Together is a real-time collaborative audio production and engineering environment built on a different computational model.

By dividing the workload between the CPU and the GPU using Vulkan, we gain a fundamentally different class of processing power. The CPU manages the session, the graph, and the network. The GPU runs the signal processing and the visualization in parallel, simultaneously, using thousands of cores designed for exactly this kind of work.

This computational headroom lets us do something current DAWs cannot: apply a real-time Fast Fourier Transform to the audio signal and decompose it into its actual components. Every sound — every voice, every instrument, every noise — is mathematically a sum of sine waves. The FFT reveals those components explicitly, making them individually addressable for the first time.

Those components exist across three dimensions:

- **Frequency** — which harmonic is present and at what pitch. The note a singer holds, the overtones above it, each as a separate object.
- **Amplitude** — the strength of each harmonic at each moment. What makes a sound bright or dark, full or thin.
- **Phase** — the timing relationship between harmonics. Invisible in a waveform, inaudible in isolation, but responsible for cancellation artifacts, the character of a room, and the way sounds combine.

Editing at this level means you are not applying a curve to the whole signal. You are addressing specific components — this harmonic, at this moment — and modifying only that. Everything else remains mathematically untouched.

What this opens

Things that exist today but are imprecise or slow

Perfect noise removal. Current tools remove noise statistically, which smears the signal and leaves artifacts. With per-component control, you identify exactly which frequency bins at which moments contain noise versus signal and remove only those. The result is clean without the underwater quality that current tools produce.

Accurate pitch correction. Auto-Tune artifacts happen because the algorithm shifts the entire signal. If you track the fundamental frequency sample by sample and correct only that component — leaving the harmonic ratios, the noise floor, and the phase relationships untouched — the correction becomes mathematically invisible. The voice sounds natural because only the one wrong thing was moved.

Fixing phase cancellation live. When two microphones capture the same source from different positions, their signals partially cancel each other at certain frequencies because the waveforms arrive out of phase. Today engineers flip the phase of the entire signal with a single button — a blunt fix that helps some frequencies and hurts others. With per-component phase control, you rotate only the frequencies that are actually canceling and leave the rest alone.

Things that are currently impossible in real time

Separating instruments from a live mixed signal. When a guitar bleeds into a vocal microphone

during a live recording, that bleed is currently permanent. But guitar and voice have different harmonic structures, different frequency envelopes, different transient shapes. A tool that understands the signal at the component level can identify which energy belongs to which source and attenuate it selectively — in real time, as it is being recorded.

Reshaping a sound without changing its pitch. A vowel sound is defined by its harmonic ratios — which overtones are emphasized relative to each other. If you can address those ratios directly, you can morph between vowel sounds continuously without touching the pitch. A singer records an "ah" and you reshape it toward an "oh" on the same note. No plugin does this in real time today.

Surgical transient control. The attack of a snare drum lasts perhaps five milliseconds. Everything that makes it feel snappy versus soft lives in that window. Current transient shapers act on the amplitude envelope of the whole signal. With time-frequency control you can reshape the spectral content of just those five milliseconds — the high-frequency crack independently of the low-frequency thud — and do it live.

Real-time physical room modeling. Convolution reverb today uses a static recording of a room. With GPU compute you can model the actual physics of sound propagating through a space and update it continuously as parameters change. The reverb responds to geometry the way actual rooms do.

Territory we have not mapped yet

Using the harmonic structure of one instrument to reshape another in real time. Not a vocoder — something more precise, where the spectral personality of a cello is applied to a synthesizer continuously as both are played.

Composing directly in the frequency domain — drawing harmonic energy in a time-frequency space rather than sequencing notes on a piano roll, and synthesizing audio from that drawing as the compositional act itself.

Taking two live signals and blending their spectral components into a third signal that belongs entirely to neither — a new kind of instrument that two performers play simultaneously.

We do not know exactly what this territory contains. That is not a weakness of the idea — it is the nature of genuinely new tools. The synthesizer opened genres nobody could have named before it existed. The most interesting question is not what we plan to build with this level of control, but what becomes possible once that control exists and musicians start using it.

Why we are building it

This project is a learning experience as much as it is a product. We want to understand audio engineering deeply — not as users of a DAW but as people who built one. We want to learn what

happens at the OS level, at the driver level, at the level of the GPU pipeline. We want to build something that reflects genuine understanding rather than surface familiarity.

We also want a tool that is actually ours — that we can use to make music together, that we control completely, that we can extend in any direction we want.

We believe there is a real gap here. The intersection of real-time collaboration, surgical signal control, and GPU-accelerated DSP does not exist in open source software. We feel a responsibility to build it openly, so that whatever we discover — whatever this level of control makes possible — belongs to anyone who wants to build on it.

Together is open source. Contributions are welcome.